



## INVESTIGANDO PROBLEMAS DE ESCALABILIDADE DE E/S NO ALGORITMO SDDP EM AMBIENTES HPC

Marcelo Barros da Cruz

Dissertação de Mestrado apresentada ao Programa de Pós-graduação em Engenharia de Sistemas e Computação, COPPE, da Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Mestre em Engenharia de Sistemas e Computação.

Orientador: Nome do Orientador Sobrenome

Rio de Janeiro  
Abril de 2026

INVESTIGANDO PROBLEMAS DE ESCALABILIDADE DE E/S NO  
ALGORITMO SDDP EM AMBIENTES HPC

Marcelo Barros da Cruz

DISSERTAÇÃO SUBMETIDA AO CORPO DOCENTE DO INSTITUTO  
ALBERTO LUIZ COIMBRA DE PÓS-GRADUAÇÃO E PESQUISA DE  
ENGENHARIA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO  
COMO PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO  
GRAU DE MESTRE EM CIÊNCIAS EM ENGENHARIA DE SISTEMAS E  
COMPUTAÇÃO.

Orientador: Nome do Orientador Sobrenome

Aprovada por: Prof. Nome do Primeiro Examinador Sobrenome  
Prof. Nome do Segundo Examinador Sobrenome  
Prof. Nome do Terceiro Examinador Sobrenome

RIO DE JANEIRO, RJ – BRASIL  
ABRIL DE 2026

Barros da Cruz, Marcelo

Investigando Problemas de Escalabilidade de E/S no Algoritmo SDDP em Ambientes HPC/Marcelo Barros da Cruz. – Rio de Janeiro: UFRJ/COPPE, 2026.

XII, 28 p.: il.; 29, 7cm.

Orientador: Nome do Orientador Sobrenome

Dissertação (mestrado) – UFRJ/COPPE/Programa de Engenharia de Sistemas e Computação, 2026.

Referências Bibliográficas: p. 27 – 28.

1. MPI-IO. 2. Lustre. 3. SDDP. 4. Computação de Alto Desempenho. 5. Escalabilidade de E/S. I. Sobrenome, Nome do Orientador. II. Universidade Federal do Rio de Janeiro, COPPE, Programa de Engenharia de Sistemas e Computação. III. Título.

*Dedico esta dissertação a [a  
**definir**].*

# Agradecimentos

**ToDp: redigir agradecimentos finais.** Coloque aqui seus agradecimentos.

Resumo da Dissertação apresentada à COPPE/UFRJ como parte dos requisitos necessários para a obtenção do grau de Mestre em Ciências (M.Sc.)

## INVESTIGANDO PROBLEMAS DE ESCALABILIDADE DE E/S NO ALGORITMO SDDP EM AMBIENTES HPC

Marcelo Barros da Cruz

Abril/2026

Orientador: Nome do Orientador Sobrenome

Programa: Engenharia de Sistemas e Computação

A escalabilidade eficiente de E/S continua sendo um desafio crítico em aplicações paralelas de dados massivos, especialmente em sistemas de alto desempenho que lidam com grandes volumes de dados. Embora avanços como o uso de MPI-IO e sistemas de arquivos paralelos como o Lustre tenham contribuído significativamente, muitas aplicações ainda enfrentam gargalos que comprometem a performance. Esse problema é particularmente evidente em modelos de otimização utilizados no setor energético, como o SDDP, que demandam acesso frequente e distribuído a grandes conjuntos de dados. A eficiência da E/S torna-se, assim, um fator decisivo para a viabilidade e o desempenho de simulações complexas em ambientes paralelos. Esta dissertação investiga os gargalos de E/S do algoritmo SDDP e propõe uma arquitetura MPI-IO baseada em MPI-IO sobre Lustre, avaliada empiricamente nos ambientes AWS (FSx for Lustre) e Santos Dumont (LNCC).

Abstract of Dissertation presented to COPPE/UFRJ as a partial fulfillment of the requirements for the degree of Master of Science (M.Sc.)

## INVESTIGATING I/O SCALABILITY ISSUES IN THE SDDP ALGORITHM ON HPC ENVIRONMENTS

Marcelo Barros da Cruz

April/2026

Advisor: Nome do Orientador Sobrenome

Department: Systems Engineering and Computer Science

Efficient I/O scalability remains a major challenge for parallel applications handling massive data volumes, particularly in high-performance computing environments. Although advances such as MPI-IO and parallel file systems like Lustre have significantly improved performance, many applications still face I/O bottlenecks that hinder scalability. This issue is particularly critical in energy sector optimization models, such as Stochastic Dual Dynamic Programming (SDDP), which require frequent and distributed access to large datasets. Therefore, optimizing I/O operations becomes essential to ensure the feasibility and efficiency of complex simulations in parallel computing systems. This dissertation investigates the I/O bottlenecks of the SDDP algorithm and proposes an MPI-IO-based architecture over Lustre, empirically evaluated on AWS (FSx for Lustre) and on the Santos Dumont supercomputer (LNCC).

# Sumário

|                                                                                    |            |
|------------------------------------------------------------------------------------|------------|
| <b>Lista de Figuras</b>                                                            | <b>x</b>   |
| <b>Lista de Tabelas</b>                                                            | <b>xi</b>  |
| <b>Lista de Abreviaturas</b>                                                       | <b>xii</b> |
| <b>1 Introdução</b>                                                                | <b>1</b>   |
| <b>2 Fundamentação Teórica</b>                                                     | <b>3</b>   |
| 2.1 MPI e protocolos de comunicação . . . . .                                      | 3          |
| 2.2 MPI-IO e operações paralelas de arquivos . . . . .                             | 3          |
| 2.3 Sistema de arquivos paralelo Lustre . . . . .                                  | 4          |
| 2.4 Adaptadores de rede em HPC: Ethernet e InfiniBand . . . . .                    | 4          |
| 2.5 Computação em nuvem para HPC: AWS . . . . .                                    | 5          |
| 2.6 O Programa Mensal da Operação (PMO) . . . . .                                  | 5          |
| 2.7 Algoritmo SDDP . . . . .                                                       | 6          |
| 2.7.1 Arquitetura atual de E/S do SDDP com um único processo<br>escritor . . . . . | 6          |
| <b>3 Paralelização dos Mecanismos de E/S do SDDP</b>                               | <b>8</b>   |
| 3.1 Uso de MPI-IO com Lustre para escalabilidade de E/S . . . . .                  | 8          |
| <b>4 Avaliações</b>                                                                | <b>11</b>  |
| 4.1 Experimento SDumont . . . . .                                                  | 12         |
| 4.1.1 Avaliação da implementação atual . . . . .                                   | 13         |
| 4.1.2 Avaliação da implementação MPI-IO . . . . .                                  | 13         |
| 4.1.3 Análise comparativa . . . . .                                                | 14         |
| 4.2 Experimento AWS . . . . .                                                      | 17         |
| 4.2.1 Avaliação da implementação atual . . . . .                                   | 18         |
| 4.2.2 Avaliação da implementação MPI-IO . . . . .                                  | 19         |
| 4.2.3 Análise comparativa . . . . .                                                | 19         |
| 4.2.4 Avaliação do impacto do número de OSTs . . . . .                             | 23         |

|          |                                   |           |
|----------|-----------------------------------|-----------|
| <b>5</b> | <b>Conclusões</b>                 | <b>26</b> |
|          | <b>Referências Bibliográficas</b> | <b>27</b> |

# Lista de Figuras

|     |                                                                                                                                                               |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Algoritmo SDDP com processo escritor. . . . .                                                                                                                 | 6  |
| 3.1 | Proposta com MPI-IO e Lustre. . . . .                                                                                                                         | 10 |
| 4.1 | Histograma das mensagens. . . . .                                                                                                                             | 12 |
| 4.2 | Banda agregada média das implementações atual e MPI-IO no Experimento SDumont. . . . .                                                                        | 15 |
| 4.3 | Decomposição do tempo médio por processo das implementações atual e MPI-IO no Experimento SDumont. . . . .                                                    | 16 |
| 4.4 | Tempo médio de bloqueio das operações de envio por tamanho de mensagem nas implementações atual e MPI-IO no Experimento SDumont (escala logarítmica). . . . . | 17 |
| 4.5 | Banda agregada média no Experimento AWS. . . . .                                                                                                              | 20 |
| 4.6 | Tempo de execução no Experimento AWS. . . . .                                                                                                                 | 22 |
| 4.7 | Tempo médio de bloqueio das operações de envio em função do tamanho das mensagens no Experimento AWS (escala logarítmica). . .                                | 22 |
| 4.8 | Experimento com diferentes nós OSTs. . . . .                                                                                                                  | 24 |
| 4.9 | Banda agregada média no experimento de OSTs. . . . .                                                                                                          | 25 |

# Lista de Tabelas

|     |                                                                                                                                                              |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Desempenho do Experimento SDumont com a implementação atual, incluindo tempos de I/O, comunicação, tempo total, <i>speedup</i> e percentual de I/O. . . . .  | 13 |
| 4.2 | Desempenho do Experimento SDumont com a implementação MPI-IO, incluindo tempos de I/O, comunicação, tempo total, <i>speedup</i> e percentual de I/O. . . . . | 14 |
| 4.3 | Desempenho do Experimento AWS com a implementação atual, incluindo tempos de I/O, comunicação, tempo total, <i>speedup</i> e percentual de I/O. . . . .      | 18 |
| 4.4 | Desempenho do Experimento AWS com a implementação MPI-IO, incluindo tempos de I/O, comunicação, tempo total, <i>speedup</i> e percentual de I/O. . . . .     | 19 |

# Lista de Abreviaturas

|         |                                                                |   |
|---------|----------------------------------------------------------------|---|
| AWS     | Amazon Web Services . . . . .                                  | 3 |
| E/S     | Entrada/Saída . . . . .                                        | 1 |
| EFA     | Elastic Fabric Adapter . . . . .                               | 3 |
| FSx     | Amazon FSx for Lustre . . . . .                                | 3 |
| HPC     | High-Performance Computing . . . . .                           | 1 |
| LNCC    | Laboratório Nacional de Computação Científica . . . . .        | 3 |
| MDS     | Metadata Server . . . . .                                      | 3 |
| MDT     | Metadata Target . . . . .                                      | 3 |
| MPI     | Message Passing Interface . . . . .                            | 1 |
| MPI-IO  | MPI Input/Output . . . . .                                     | 3 |
| ONS     | Operador Nacional do Sistema Elétrico . . . . .                | 3 |
| OSS     | Object Storage Server . . . . .                                | 3 |
| OST     | Object Storage Target . . . . .                                | 3 |
| PMO     | Programa Mensal da Operação . . . . .                          | 3 |
| RDMA    | Remote Direct Memory Access . . . . .                          | 3 |
| SDDP    | Stochastic Dual Dynamic Programming . . . . .                  | 1 |
| SIN     | Sistema Interligado Nacional . . . . .                         | 3 |
| SINAPAD | Sistema Nacional de Processamento de Alto Desempenho . . . . . | 3 |

# Capítulo 1

## Introdução

O aumento da complexidade e do volume de dados em aplicações científicas e de engenharia tem tornado a computação paralela uma ferramenta indispensável para atender às demandas de desempenho TODO: AUTOR (2024). Entretanto, operações de entrada e saída (E/S) continuam sendo um dos principais gargalos em ambientes de alto desempenho (HPC), particularmente em aplicações que processem dados massivos THAKUR *et al.* (1999). Nesse contexto, o uso de bibliotecas como o MPI-IO, padronizada pelo MPI-2, combinada com sistemas de arquivos paralelos como o Lustre, tornou-se uma abordagem fundamental para lidar com a escalabilidade e a eficiência no acesso aos dados TODO (2010a); BRAAM (2002).

Paralelamente, a crescente inserção de fontes renováveis na matriz energética, como solar e eólica, tem introduzido novas dificuldades na formulação e resolução de problemas de otimização associados à operação dos sistemas elétricos. A variabilidade e incerteza dessas fontes requerem modelos computacionais mais complexos e dependentes de grandes volumes de dados, cuja execução eficiente depende tanto do paralelismo computacional quanto de soluções eficazes de E/S TODO (2010b, 2019).

**ToDp: precisamos expandir esse parágrafo, explicar o que implica essa mudança de paradigma na simulação tanto para o uso de recursos computacionais como para o uso dos resultados pelos usuários do SDDP.**

Assim como outras aplicações baseadas em tarefas independentes, o modelo SDDP (*Stochastic Dual Dynamic Programming*) é amplamente utilizado para planejamento e despacho hidrotérmico em diversos países. À medida que os modelos se tornam mais detalhados, em resolução horária, e a quantidade de cenários aumenta, a execução do SDDP passa a demandar não apenas alto poder computacional, mas também operações intensivas de E/S PEREIRA e PINTO (1991).

Nesse contexto, a análise do desempenho do MPI-IO sobre sistemas de arquivos paralelos como o Lustre, aplicada a esse tipo de aplicação, representa uma contribuição significativa para o desenvolvimento de soluções mais robustas e escaláveis em

ambientes HPC.

**ToDp:** Apresentar organização da dissertação ao final da introdução (capítulos 2–5).

# Capítulo 2

## Fundamentação Teórica

### 2.1 MPI e protocolos de comunicação

O *Message Passing Interface* (MPI) é o padrão para programação paralela baseada em troca de mensagens, sendo amplamente utilizado em *clusters* e supercomputadores. Um aspecto essencial da sua implementação são os protocolos de comunicação: o *eager*, que envia mensagens pequenas de forma imediata, sem necessidade de confirmação do receptor, favorecendo baixa latência; e o *rendezvous*, que aguarda o *handshake* do receptor antes de transmitir mensagens maiores, reduzindo *overhead* de memória e aumentando a previsibilidade do desempenho em cargas intensivas MESSAGE PASSING INTERFACE FORUM (2023). A escolha entre os protocolos depende fortemente do tamanho da mensagem e da disponibilidade de *buffers*: enquanto o *eager* utiliza *buffers* internos para armazenar temporariamente dados até que o receptor os processe, o *rendezvous* elimina essa necessidade para mensagens grandes, evitando consumo excessivo de memória e garantindo maior estabilidade em aplicações de larga escala TODO (1999, 2005a).

### 2.2 MPI-IO e operações paralelas de arquivos

O MPI também estende suas funcionalidades à entrada e saída de dados por meio da especificação MPI-IO MESSAGE PASSING INTERFACE FORUM (1997), que define uma API padronizada para operações paralelas de arquivos. Essa interface possibilita que múltiplos processos acessem simultaneamente grandes volumes de dados, permitindo otimizações como *collective I/O*, técnicas de agregação e *data sieving*, que reduzem o número de operações pequenas e melhoram a eficiência do uso do sistema de arquivos. Entre as chamadas mais utilizadas, destacam-se `MPI_File_write_at`, que realiza uma escrita independente em um deslocamento específico do arquivo, permitindo controle preciso sobre onde os dados serão gravados, e `MPI_`

`File_write_all`, que executa uma operação coletiva em que todos os processos participantes escrevem de forma coordenada. Enquanto a primeira é adequada para acessos irregulares ou não sincronizados, a segunda explora a comunicação coletiva para reduzir *overhead* de metadados e alinhar requisições, sendo mais eficiente em ambientes com sistemas de arquivos distribuídos como o Lustre.

## 2.3 Sistema de arquivos paralelo Lustre

O sistema de arquivos paralelo Lustre é utilizado na maioria dos supercomputadores da última lista do Top500 TODO (2025). Projetado especificamente para cargas massivas de HPC, ele distribui os dados em *Object Storage Targets* (OSTs) e os metadados em *Metadata Servers* (MDS), permitindo acesso paralelo e balanceado a múltiplos clientes. Além disso, o Lustre utiliza a técnica de *striping*, na qual um arquivo é dividido em blocos (*chunks* ou *stripes*) distribuídos entre diferentes OSTs. O tamanho do *stripe* e o número de OSTs utilizados podem ser configurados de acordo com o padrão de acesso da aplicação, influenciando diretamente no *throughput* e na escalabilidade: *stripes* maiores tendem a favorecer grandes operações sequenciais, enquanto *stripes* menores podem beneficiar acessos concorrentes e distribuídos BRAAM (2002). Graças a essa flexibilidade e alto desempenho, o Lustre tornou-se amplamente adotado em supercomputadores listados no TOP500, constituindo um elemento-chave para aplicações científicas e industriais que exigem desempenho extremo em E/S.

## 2.4 Adaptadores de rede em HPC: Ethernet e InfiniBand

Os adaptadores de rede são componentes críticos para o desempenho em sistemas de computação de alto desempenho, sendo a Ethernet e o InfiniBand as tecnologias mais utilizadas. A Ethernet, tradicionalmente projetada para redes locais corporativas, evoluiu para padrões de alta velocidade, como 40/100/400 Gbps, oferecendo elevada taxa de transferência (*bandwidth*), embora apresente latências relativamente maiores, frequentemente na ordem de dezenas de microssegundos. Já o InfiniBand foi desenvolvido especificamente para ambientes de HPC, combinando altas larguras de banda — que podem ultrapassar 400 Gbps em implementações modernas — com latências extremamente baixas, tipicamente abaixo de 2  $\mu$ s, graças a recursos como comunicação RDMA (*Remote Direct Memory Access*). Essa diferença torna o InfiniBand preferido em supercomputadores e *clusters* científicos, onde a baixa latência na troca de mensagens é tão relevante quanto a banda disponível, enquanto

a Ethernet, pelo menor custo e ampla compatibilidade, mantém espaço em sistemas híbridos e *datacenters* TODO (2003, 2005b).

## 2.5 Computação em nuvem para HPC: AWS

A Amazon Web Services (AWS) tem se consolidado como uma das principais plataformas de computação em nuvem para aplicações de *High Performance Computing* (HPC), oferecendo recursos sob demanda que permitem a execução de cargas de trabalho intensivas em computação e E/S. Entre os serviços disponíveis, destacam-se as instâncias otimizadas para HPC, como as famílias C, H e P, que fornecem alto poder de processamento aliado a interconexões de baixa latência por meio do *Elastic Fabric Adapter* (EFA), permitindo a utilização de bibliotecas como MPI em escala. Para atender aplicações intensivas em dados, a AWS integra sistemas de armazenamento de alto desempenho, como o Amazon FSx for Lustre, que viabiliza acesso paralelo e em larga escala, semelhante a sistemas de arquivos utilizados em supercomputadores tradicionais. Essa combinação de infraestrutura elástica, rede de alto desempenho e suporte a ferramentas amplamente usadas em HPC torna a nuvem da AWS uma alternativa competitiva frente a *clusters* locais, especialmente em cenários que demandam escalabilidade e flexibilidade TODO (2010c, 2020).

## 2.6 O Programa Mensal da Operação (PMO)

No setor elétrico brasileiro, o Programa Mensal da Operação (PMO) é um documento técnico elaborado pelo Operador Nacional do Sistema Elétrico (ONS) que estabelece as diretrizes de curto prazo para a operação do Sistema Interligado Nacional (SIN). O PMO contempla projeções de carga, disponibilidade de geração, restrições de transmissão e condições hidrológicas, servindo como referência para a programação da operação hidrotérmica de forma a garantir o equilíbrio entre oferta e demanda de energia elétrica. Além disso, define metas de despacho de usinas hidrelétricas e termelétricas, contribuindo para a segurança energética e para a otimização do uso dos recursos disponíveis. Por sua relevância, o PMO orienta não apenas as decisões operativas do ONS, mas também agentes do setor, incluindo geradores, comercializadores e distribuidores, sendo um instrumento fundamental para a gestão integrada e eficiente do SIN OPERADOR NACIONAL DO SISTEMA ELÉTRICO (2023).

## 2.7 Algoritmo SDDP

**ToDo: SDDP deve ser detalhado aqui, bem como sua E/S atual. ToDp: colocar figura cenários × estágios.**

A aplicação SDDP é um modelo matemático baseado em MPI (*Message Passing Interface*), projetado para resolver diversos problemas de forma paralela. Cada *rank* de computação é responsável pela execução de uma instância do problema, enquanto um único processo concentra todas as atividades de entrada/saída (E/S). Historicamente, essa abordagem não apresentava limitações de escalabilidade. No entanto, com o aumento da complexidade dos algoritmos e do detalhamento dos dados — o que resultou em um volume significativamente maior de informações — começaram a surgir gargalos relacionados à escalabilidade do sistema.

### 2.7.1 Arquitetura atual de E/S do SDDP com um único processo escritor

Na arquitetura atual do SDDP, apenas um *rank* MPI (o *rank* 1) é designado para realizar todas as operações de E/S do sistema. Cada nó de computação, após resolver sua respectiva tarefa, envia os resultados por meio de um *buffer* de dados utilizando a função `MPI_Send`. Esse *buffer* é então recebido pelo processo escritor através de uma chamada `MPI_Recv` e, posteriormente, os dados são gravados em um sistema de arquivos distribuído. A Figura 2.1 ilustra o funcionamento dessa arquitetura.

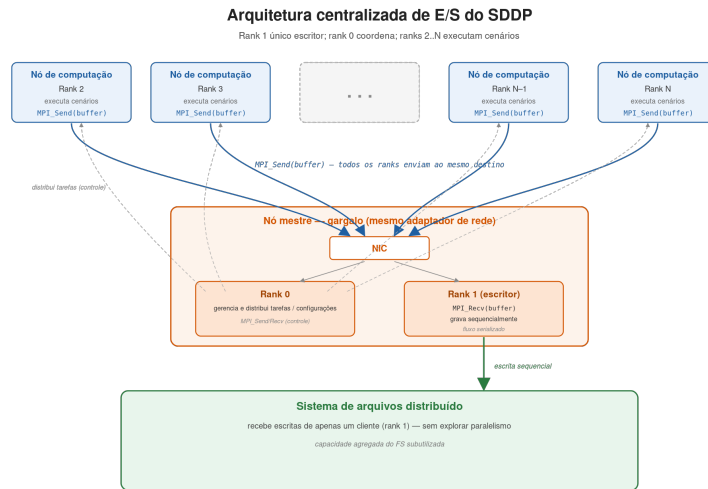


Figura 2.1: Algoritmo SDDP com processo escritor.

Independentemente do número de nós computacionais utilizados, apenas um processo é responsável pelas operações de E/S. Cada nó utiliza sua banda efetiva para enviar os *buffers* de dados, enquanto o processo escritor utiliza sua banda

para recebê-los. A primeira hipótese para as limitações de escalabilidade nessa arquitetura está relacionada ao adaptador de rede do servidor que executa o processo mestre.

Adicionalmente, o *rank* 0 é responsável pelo gerenciamento e controle da aplicação, como a distribuição de tarefas para os demais *ranks* e o envio/recebimento das configurações do sistema. Essas atividades também utilizam o mesmo adaptador de rede empregado na recepção dos *buffers* de dados. Outra hipótese é que o processo escritor opera continuamente em um fluxo sequencial de recepção e escrita dos dados, o que pode representar um gargalo adicional, sobretudo em sistemas com alto volume de dados. Por fim, se houver atrasos na recepção ou na gravação dos *buffers* de dados por parte do processo mestre, os nós computacionais podem ser obrigados a aguardar, gerando sobrecarga no envio dos *buffers* e aumentando o custo computacional da execução paralela.

## Capítulo 3

# Paralelização dos Mecanismos de E/S do SDDP

Este capítulo apresenta a proposta de paralelização dos mecanismos de entrada e saída (E/S) do SDDP. A solução substitui o modelo centralizado, no qual um único *rank* recebe os *buffers* de todos os processos e realiza a escrita, por uma organização baseada em MPI-IO sobre o sistema de arquivos Lustre. Nessa abordagem, cada *rank* grava diretamente seus próprios resultados em regiões independentes do espaço de saída, evitando a concentração das operações de E/S em um único processo.

O ponto central da proposta é explorar, simultaneamente, o paralelismo da aplicação MPI e o paralelismo do sistema de arquivos. Como os resultados produzidos por diferentes *ranks* são independentes, suas escritas podem ser posicionadas previamente e executadas em paralelo. No Lustre, os arquivos são distribuídos em *stripes* entre diferentes *Object Storage Targets* (OSTs); assim, múltiplas escritas independentes podem ser atendidas por OSTs distintos, aumentando a largura de banda agregada e reduzindo a contenção observada no modelo com escritor centralizado.

Dessa forma, a proposta reduz dois gargalos da arquitetura original. O primeiro é o gargalo de comunicação, pois os dados deixam de ser enviados ao processo escritor antes de serem persistidos. O segundo é o gargalo de armazenamento, pois a escrita passa a ser distribuída entre os recursos do sistema de arquivos paralelo, em vez de ser serializada por um único *rank*. A Figura 3.1 sintetiza essa nova organização.

### 3.1 Uso de MPI-IO com Lustre para escalabilidade de E/S

A atual arquitetura da aplicação SDDP, baseada na biblioteca MPI, utiliza apenas um processo — tipicamente o *rank* 1 — para executar todas as operações de entrada/saída (E/S). Embora esse modelo tenha sido suficiente em versões anteriores

da aplicação, o aumento no volume de dados, decorrente de maior detalhamento e da complexidade algorítmica, passou a gerar gargalos severos de desempenho e escalabilidade. Como hipótese para mitigar essas limitações, propõe-se a substituição do modelo atual de E/S por uma abordagem MPI-IO, baseada na interface MPI-IO em conjunto com o sistema de arquivos distribuído Lustre.

A interface MPI-IO, parte do padrão MPI-2, fornece mecanismos para a realização de operações de E/S por múltiplos processos MPI sobre arquivos compartilhados. Diferentemente do modelo tradicional de comunicação ponto a ponto (`MPI_Send/MPI_Recv`), o MPI-IO permite que cada processo realize operações diretas de escrita em arquivos, sem a necessidade de coordenação com um processo único de escrita THAKUR *et al.* (1999). Essa flexibilidade possibilita que os próprios processos responsáveis pela computação escrevam seus resultados diretamente em disco, utilizando *offsets* distintos e evitando conflitos de acesso. Isso é particularmente relevante em aplicações como o SDDP, em que os dados produzidos por cada *rank* são independentes e podem ser gravados em posições previamente determinadas de um arquivo compartilhado, ou mesmo em arquivos separados.

Aliado ao MPI-IO, propõe-se o uso do sistema de arquivos Lustre, amplamente adotado em ambientes de computação de alto desempenho (HPC). O Lustre foi projetado para oferecer alto rendimento e paralelismo no acesso a arquivos, distribuindo os dados entre múltiplos servidores de armazenamento (OSTs — *Object Storage Targets*) BRAAM (2002). Essa característica torna o Lustre particularmente eficiente em cenários onde múltiplos nós executam operações simultâneas de escrita, como se espera com a integração do MPI-IO. A combinação das duas tecnologias permite que os *ranks* MPI realizem operações de E/S em paralelo, aproveitando ao máximo a largura de banda disponível no sistema e reduzindo significativamente os gargalos causados pela concentração da escrita.

Espera-se que essa nova arquitetura traga ganhos substanciais de desempenho. Primeiramente, ao eliminar o processo escritor único, a aplicação poderá distribuir dinamicamente a carga de E/S entre os próprios processos computacionais, reduzindo a sobrecarga sobre o adaptador de rede do nó mestre. Em segundo lugar, a redução da dependência de *buffers* intermediários e de sincronizações entre *ranks* minimiza a latência entre o término da computação e a persistência dos resultados. Além disso, a utilização do paralelismo nativo do Lustre deverá resultar em um melhor aproveitamento dos recursos de armazenamento, tornando o sistema mais escalável à medida que aumenta o número de nós. Por fim, essa abordagem alinha-se às boas práticas adotadas em aplicações científicas de larga escala, promovendo maior portabilidade, eficiência e adaptabilidade a infraestruturas HPC modernas. A Figura 3.1 descreve o funcionamento da proposta.

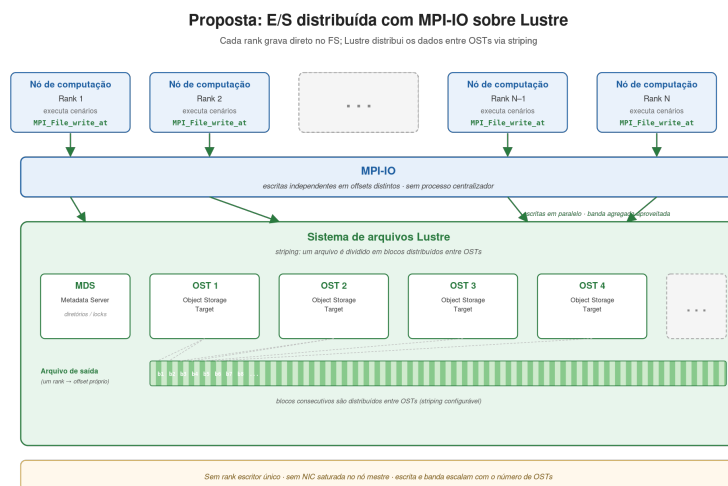


Figura 3.1: Proposta com MPI-IO e Lustre.

# Capítulo 4

## Avaliações

Este capítulo apresenta a avaliação comparativa entre a implementação atual de entrada e saída (E/S) e a implementação proposta, com o objetivo de investigar ganhos de desempenho e melhorias de escalabilidade, principalmente em ambientes de computação em nuvem. A análise concentra-se em métricas de tempo de execução, tempo de comunicação, largura de banda e tempo de bloqueio das operações de E/S, considerando o impacto das estratégias adotadas para operações paralelas de E/S em cenários de alto desempenho THAKUR *et al.* (1999); TODO (2009).

A base de dados utilizada nos experimentos corresponde ao PMO/PAR de outubro de 2023. Para a etapa de simulação horária, que constitui o foco principal deste estudo, foram considerados 1.536 cenários em ambos os experimentos (AWS e SDumont), com três estágios mensais.

Como cada *rank* MPI apresentou consumo de memória residente superior a 8 GB, optou-se, em ambos os experimentos, por mapear apenas um *rank* por núcleo físico — isto é, sem o uso do *hyper-threading*. Essa decisão evita que dois *ranks* compartilhando o mesmo núcleo físico disputem a mesma hierarquia de memória, situação em que o tempo de simulação tende a ser dominado por *cache thrashing* e por pressão sobre o subsistema de memória NIKOLOPOULOS (2004). O número de *ranks* por nó foi, portanto, fixado no número de núcleos físicos da instância correspondente.

Os experimentos conduzidos neste trabalho caracterizam um cenário de *strong scaling*, no qual o tamanho do problema permanece fixo enquanto o número de nós computacionais é progressivamente aumentado AMDAHL (1967). Nesse contexto, o acréscimo de recursos implica na redução do volume de trabalho por nó, sendo esperado, idealmente, que o tempo total de execução diminua de forma aproximadamente linear com o aumento do paralelismo. No entanto, essa tendência teórica pode não se concretizar na prática devido à presença de gargalos sistêmicos, especialmente aqueles relacionados às operações de entrada e saída (E/S) e à comunicação entre processos. À medida que o número de nós cresce, a intensificação da concorrência por recursos compartilhados — como o sistema de arquivos e a rede —

pode introduzir sobrecargas adicionais, limitando a eficiência do escalonamento e reduzindo os ganhos esperados de desempenho.

O tamanho das mensagens foi considerado um fator determinante para a análise de desempenho de E/S paralela. Em cada resolução de um problema, o padrão de mensagens permanece consistente, uma vez que a carga de saída é fixa por cenário. A Figura 4.1 apresenta a distribuição do tamanho das mensagens para um único cenário representativo; entretanto, esse comportamento se repete de forma consistente nos demais cenários avaliados. Observa-se que uma parcela significativa das mensagens concentra-se em tamanhos reduzidos, em torno de 1 kB, o que pode comprometer a escalabilidade, já que operações com granularidade fina tendem a aumentar a sobrecarga de metadados e a subutilizar a largura de banda disponível THAKUR *et al.* (1999); TODO (2009). Adicionalmente, será avaliado o tempo de envio das mensagens agrupadas por faixas de tamanho, de forma a analisar o impacto da granularidade das operações no desempenho da comunicação e da E/S.

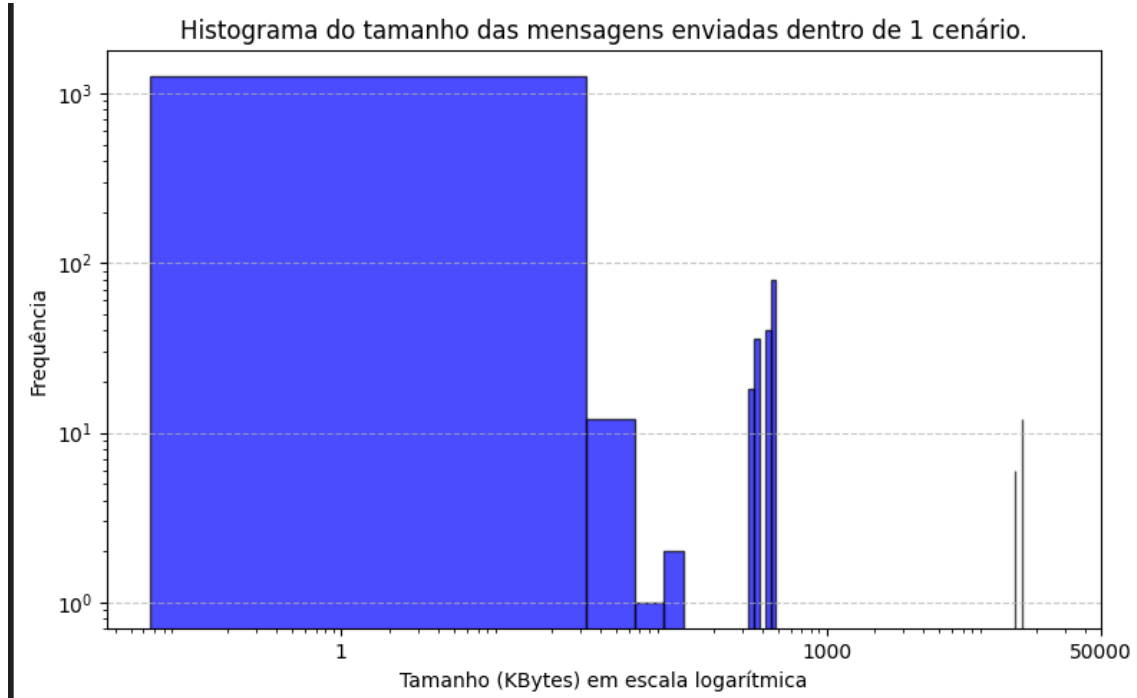


Figura 4.1: Histograma das mensagens.

## 4.1 Experimento SDumont

O Experimento SDumont corresponde ao supercomputador Santos Dumont (SDumont), adquirido junto à empresa francesa EVIDEN/ATOS/BULL, que está localizado na sede do Laboratório Nacional de Computação Científica (LNCC), em Petrópolis-RJ, atuando como nó central (Tier-0) do Sistema Nacional de Processamento de Alto Desempenho (SINAPAD). Nesse experimento foram utilizados até

32 nós de processamento, cada um equipado com dois processadores Intel Xeon Cascade Lake Gold 6252, totalizando 24 núcleos físicos por nó, com suporte a *hyper-threading*, o que resulta em 48 processadores lógicos disponíveis. Cada nó conta ainda com 384 GB de memória e interconexão de alta velocidade baseada em InfiniBand EDR de 100 Gb/s, projetada para reduzir a latência e aumentar a eficiência em aplicações paralelas de larga escala. Nos experimentos, foi utilizada a biblioteca MPICH2 versão 3.2 e um sistema de arquivos paralelo Lustre v2.12 implementado através do ClusterStor L300 da Cray/HPE. Essa implementação do Lustre é composta por 1 nó MDS (com 1 MDT) e 6 nós OSSs (cada um servindo 1 OST) e *stripes* de 1 MB, disponibilizando um espaço total de armazenamento de 1,1 PB. Pelos motivos discutidos na introdução do capítulo, também aqui se configurou um *rank* por núcleo físico, totalizando 24 cenários simultâneos por nó.

#### 4.1.1 Avaliação da implementação atual

Esta subseção analisa o desempenho da arquitetura atual de E/S do SDDP no Experimento SDumont. A Tabela 4.1 sumariza os resultados, considerando 1.536 cenários, três estágios mensais e 24 *ranks* por nó.

O tempo total de execução não apresenta tendência monotônica de redução com o aumento do número de nós: cai de 8.444 s em 2 nós para 5.921 s em 4 nós, mas volta a crescer em 8 e 16 nós, atingindo 7.805 s nessa última escala. A banda agregada cai consistentemente, de 4,57 Gb/s em 2 nós para 0,84 Gb/s em 16 nós. O percentual do tempo dedicado à comunicação se mantém em torno de 11–17% em todas as escalas. Esse comportamento caracteriza um regime degradado sob *strong scaling*, no qual a contenção do processo escritor mais do que compensa os ganhos de paralelismo computacional.

Tabela 4.1: Desempenho do Experimento SDumont com a implementação atual, incluindo tempos de I/O, comunicação, tempo total, *speedup* e percentual de I/O.

| Nós | Banda<br>(Gb/s) | I/O<br>(s)        | Comunicação<br>(s) | Total<br>(s)      | Speedup | % I/O  |
|-----|-----------------|-------------------|--------------------|-------------------|---------|--------|
| 2   | 4,57 ± 0,56     | 894,81 ± 341,58   | 932,59 ± 48,27     | 9 376,80 ± 349,11 | 1,00×   | 9,54%  |
| 4   | 2,04 ± 0,54     | 2 116,61 ± 482,68 | 921,08 ± 55,50     | 6 841,99 ± 479,20 | 1,37×   | 30,94% |
| 8   | 1,52 ± 1,01     | 4 013,54 ± 220,86 | 1 014,67 ± 36,82   | 6 981,25 ± 163,40 | 1,34×   | 57,49% |
| 16  | 0,84 ± 0,75     | 6 602,61 ± 671,38 | 1 277,79 ± 30,77   | 9 082,37 ± 174,91 | 1,03×   | 72,70% |

#### 4.1.2 Avaliação da implementação MPI-IO

Esta subseção analisa o desempenho da arquitetura proposta no Experimento SDumont, baseada em escritas independentes com MPI-IO sobre o sistema de arquivos

Lustre operado pelo ClusterStor L300. A Tabela 4.2 consolida os resultados.

Em relação à implementação atual, o tempo total de execução cai modestamente em 4 e 8 nós — de 9.375 s em 2 nós para 5.580 s em 4 nós e 5.366 s em 8 nós (reduções de cerca de 40% e 43%, respectivamente) — mas volta a crescer em 16 nós, atingindo 6.407 s. A banda agregada do *layer* de escrita permanece modesta, oscilando entre 1,6 e 2,8 Gb/s ao longo das escalas avaliadas, indicando que a implementação MPI-IO, ainda que elimine o ponto único de contenção do processo escritor, sofre com alta variabilidade. Os desvios padrão expressivos a partir de 8 nós (da ordem do próprio valor médio para o tempo de E/S) confirmam essa instabilidade — comportamento típico de saturação do sistema de metadados / coletivas no Lustre quando muitos processos abrem e fecham arquivos simultaneamente.

Tabela 4.2: Desempenho do Experimento SDumont com a implementação MPI-IO, incluindo tempos de I/O, comunicação, tempo total, *speedup* e percentual de I/O.

| Nós | Banda<br>(Gb/s) | I/O<br>(s)                | Comunicação<br>(s)     | Total<br>(s)              | Speedup      | % I/O  |
|-----|-----------------|---------------------------|------------------------|---------------------------|--------------|--------|
| 2   | $1,92 \pm 0,22$ | $1\,505,19 \pm 338,26$    | $968,96 \pm 134,71$    | $10\,343,93 \pm 511,69$   | $1,00\times$ | 14,55% |
| 4   | $2,76 \pm 0,53$ | $1\,186,44 \pm 414,20$    | $850,55 \pm 177,58$    | $6\,430,57 \pm 489,98$    | $1,61\times$ | 18,45% |
| 8   | $2,08 \pm 0,63$ | $1\,884,36 \pm 2\,092,62$ | $976,98 \pm 213,86$    | $6\,342,98 \pm 2\,424,11$ | $1,63\times$ | 29,71% |
| 16  | $1,63 \pm 0,56$ | $1\,958,12 \pm 2\,025,19$ | $1\,185,57 \pm 487,98$ | $7\,592,64 \pm 2\,840,08$ | $1,36\times$ | 25,79% |

### 4.1.3 Análise comparativa

Os resultados das duas subseções anteriores são consolidados visualmente nas figuras a seguir, que apresentam, lado a lado, os perfis de banda agregada, tempo de execução e tempo de bloqueio de envio em função do número de nós para as duas implementações no Experimento SDumont.

A Figura 4.2 compara a banda agregada média obtida pela implementação atual e pela implementação MPI-IO. Em 2 nós, a implementação atual apresenta a maior banda observada, com cerca de 4,6 Gb/s, enquanto a versão MPI-IO atinge aproximadamente 1,9 Gb/s. A partir de 4 nós, entretanto, a implementação MPI-IO passa a apresentar maior banda agregada que a implementação atual, alcançando cerca de 2,8 Gb/s em 4 nós, 2,1 Gb/s em 8 nós e 1,6 Gb/s em 16 nós.

Esse comportamento indica que, em pequena escala, o custo adicional da abordagem MPI-IO ainda não é compensado pelo paralelismo de escrita. Com o aumento do número de nós, a contenção sobre o processo escritor da implementação atual torna-se dominante, levando à queda progressiva da banda agregada. A implementação MPI-IO reduz esse gargalo ao distribuir as operações de E/S, mas também apresenta queda de desempenho nas maiores configurações, indicando que a escalabilidade no

SDumont é limitada por fatores adicionais da infraestrutura de armazenamento e pela concorrência entre os processos.

Um possível gargalo adicional no Experimento SDumont está associado ao subsistema de metadados do Lustre. A configuração utilizada possui um único MDS/MDT, responsável por manter o espaço de nomes, atributos, permissões, layouts e alocação inicial dos objetos de arquivos; embora o fluxo de dados seja transferido diretamente entre clientes e OSTs após a abertura dos arquivos, operações como criação, abertura, fechamento e atualização de metadados continuam passando pelo MDT e pelo gerenciador distribuído de locks do Lustre BRAAM (2002); OPENSFS AND EOFS (2025). Assim, quando diversos *ranks* executam operações de E/S concorrentes, a contenção por *metadata locks* e a serialização parcial de operações no MDT podem limitar a escalabilidade, explicando a variabilidade e a queda de banda observadas mesmo após a remoção do processo escritor centralizado.

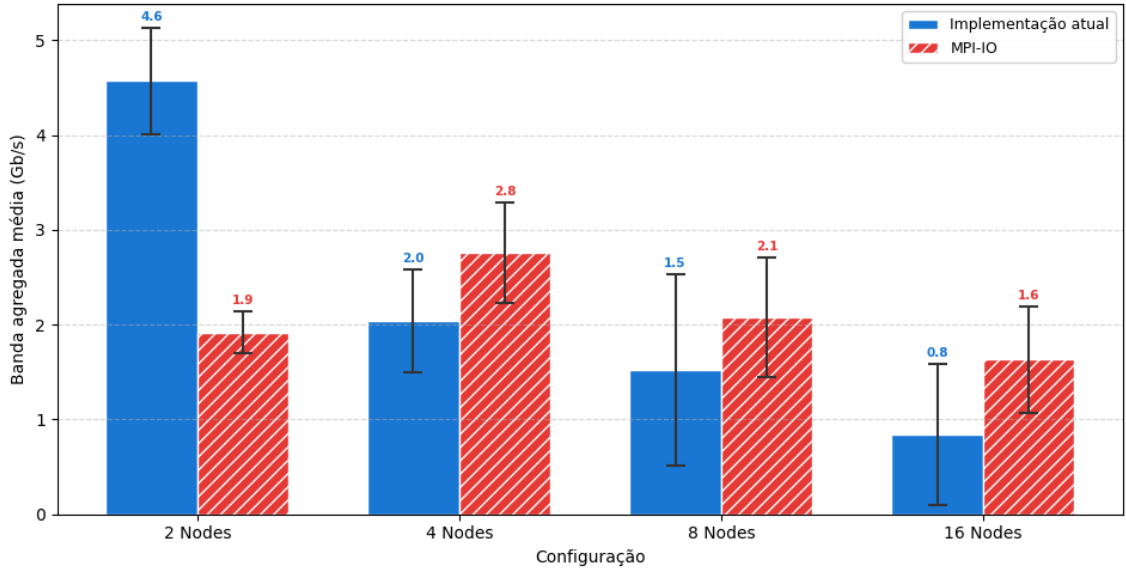


Figura 4.2: Banda agregada média das implementações atual e MPI-IO no Experimento SDumont.

A Figura 4.3 detalha o tempo médio por processo no Experimento SDumont, separando as parcelas de computação, comunicação, E/S e operações coletivas MPI. Em 2 nós, a implementação MPI-IO apresenta tempo total superior ao da implementação atual, pois o custo das operações coletivas e da própria camada MPI-IO ainda pesa de forma significativa. Nas configurações com 4, 8 e 16 nós, a versão MPI-IO reduz o tempo total em relação à implementação atual, com ganhos mais evidentes nas maiores escalas.

Na implementação atual, a parcela de E/S cresce fortemente com o aumento do número de nós, chegando a representar a maior parte do tempo total em 16 nós. Esse comportamento é compatível com a saturação do processo escritor centralizado.

Na implementação MPI-IO, o tempo de E/S é redistribuído entre os processos e permanece mais controlado, embora o custo das operações coletivas passe a compor uma fração visível do tempo total. Assim, a figura evidencia que a abordagem proposta reduz o gargalo centralizado de escrita, mas não elimina todos os custos de coordenação associados ao uso de MPI-IO no SDumont.

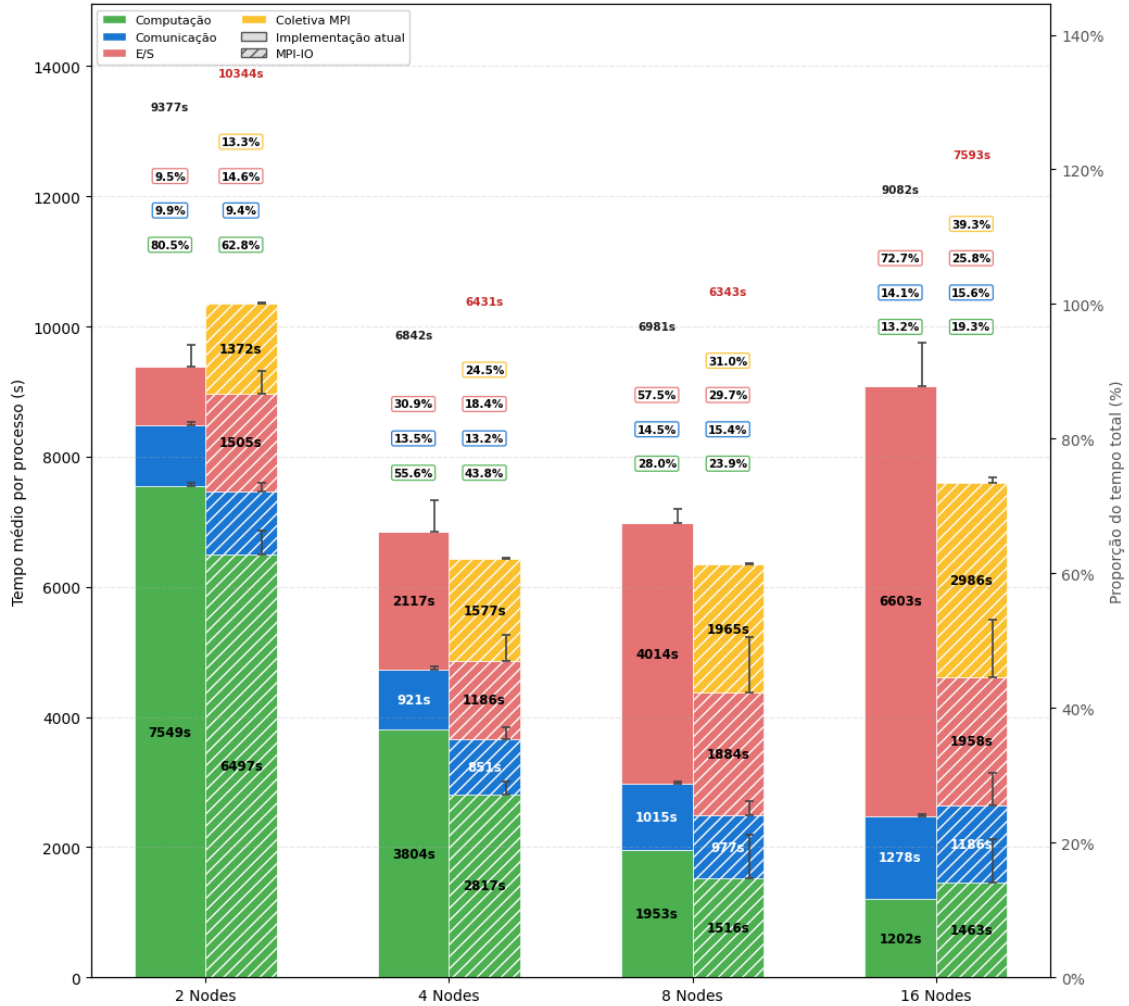


Figura 4.3: Decomposição do tempo médio por processo das implementações atual e MPI-IO no Experimento SDumont.

A Figura 4.4 apresenta o tempo médio de bloqueio das operações de envio em função do tamanho das mensagens no Experimento SDumont, em escala logarítmica. A comparação mostra comportamentos distintos conforme o tamanho da mensagem. Para mensagens muito pequenas, a implementação atual apresenta tempos de envio menores, pois evita o custo fixo das chamadas MPI-IO. Para mensagens maiores, especialmente nas classes de 100 MB e 50 MB, a implementação MPI-IO reduz substancialmente o tempo de bloqueio em relação ao modelo centralizado.

Esse resultado mostra que o benefício da escrita paralela torna-se mais relevante quando o volume transferido por operação é maior. Na implementação atual,

mensagens grandes pressionam o processo escritor e elevam o tempo de bloqueio à medida que o número de nós cresce. Na implementação MPI-IO, a distribuição das escritas pelo sistema de arquivos Lustre reduz essa contenção, ainda que mensagens pequenas e intermediárias possam sofrer impacto do custo fixo de coordenação.

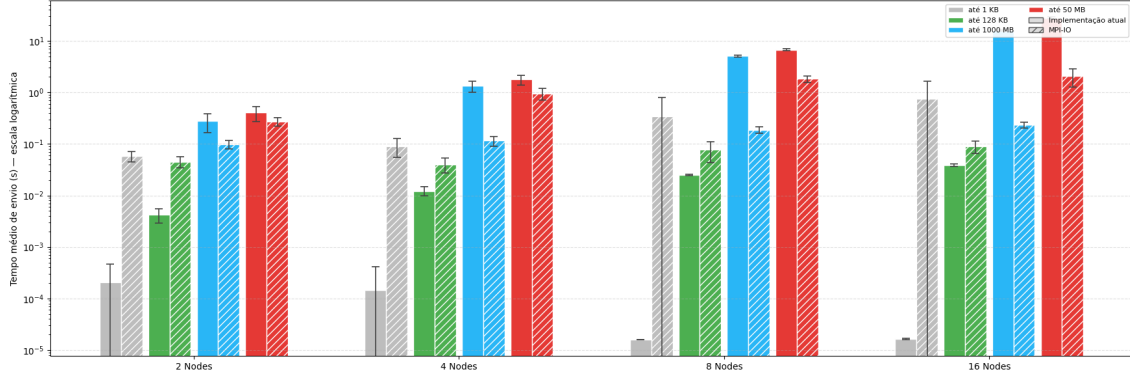


Figura 4.4: Tempo médio de bloqueio das operações de envio por tamanho de mensagem nas implementações atual e MPI-IO no Experimento SDumont (escala logarítmica).

Os resultados apresentados nesta seção demonstram que a adoção da abordagem MPI-IO com escritas independentes elimina de forma efetiva o principal gargalo da arquitetura original, associado à concentração das operações de E/S. Observa-se uma redução consistente nos tempos de comunicação por processo, bem como um comportamento mais estável do tempo total de execução à medida que o número de nós é aumentado, sem a ocorrência de degradações significativas em cenários de maior escala. Adicionalmente, a redução do tempo de bloqueio das operações de E/S contribui para uma melhor utilização dos recursos computacionais. Esses resultados, verificados de forma consistente nos Experimentos AWS e SDumont, evidenciam que a estratégia proposta melhora significativamente a escalabilidade da aplicação, tornando-a mais adequada para execuções em ambientes de computação de alto desempenho com elevado grau de paralelismo e volume de dados.

## 4.2 Experimento AWS

O Experimento AWS foi configurado na nuvem da Amazon Web Services (AWS), utilizando 32 nós da instância do tipo r7i.12xlarge. Essa instância é baseada na quarta geração de processadores Intel Xeon Scalable (Sapphire Rapids 8488C), operando a 3,2 GHz, e disponibiliza 24 núcleos físicos com suporte a *hyper-threading*, totalizando 48 processadores lógicos. O sistema conta ainda com 384 GB de memória DDR5, que oferece maior largura de banda em relação às gerações anteriores, e um adaptador de rede Ethernet com capacidade de até 18,75 Gbps, projetado

para cargas de trabalho intensivas em comunicação. Essa configuração enquadra-se na categoria de instâncias otimizadas para memória da AWS (*memory-optimized*), adequada a aplicações que combinam elevada demanda de processamento paralelo com grandes volumes de dados em memória — perfil característico do SDDP, em que cada *rank* MPI chega a consumir mais de 8 GB de memória residente. Nos experimentos, utilizou-se a biblioteca MPICH2 versão 3.2 e um sistema de arquivos paralelo Lustre versão 2.12 de 12 TB, com 1 MDS de 350 GB e 10 OSTs de 1,165 TB e *stripes* de 1 MB, disponibilizado por meio do serviço Amazon FSx for Lustre. Pelos motivos discutidos na introdução do capítulo, configurou-se um *rank* por núcleo físico, totalizando 24 cenários simultâneos por nó INTEL CORPORATION (2023); AMAZON WEB SERVICES (2024).

#### 4.2.1 Avaliação da implementação atual

Esta subseção analisa o desempenho da arquitetura atual de E/S do SDDP no Experimento AWS. A Tabela 4.3 sumariza os resultados obtidos com a base de dados PMO/PAR de outubro de 2023, considerando 1.536 cenários, três estágios mensais e 24 *ranks* por nó.

Observa-se que o tempo total de execução cai com o aumento do número de nós até 16 nós, mas estabiliza-se em torno de  $1,71 \times 10^3$  s a partir desse ponto, sem ganho adicional ao se passar para 32 nós — caracterizando o regime saturado típico de um cenário de *strong scaling* limitado pela E/S. A banda agregada degrada-se de 60,5 Gb/s em 2 nós para 3,4 Gb/s em 32 nós (mais de uma ordem de grandeza), e o percentual do tempo dedicado à comunicação cresce de cerca de 2% em 2 nós para 17,9% em 32 nós. Esses números são coerentes com o diagnóstico de que o adaptador de rede do nó mestre se torna ponto de contenção à medida que aumenta o número de processos emissores concorrentes.

Tabela 4.3: Desempenho do Experimento AWS com a implementação atual, incluindo tempos de I/O, comunicação, tempo total, *speedup* e percentual de I/O.

| Nós | Banda<br>(Gb/s)  | I/O<br>(s)          | Comunicação<br>(s) | Total<br>(s)           | Speedup | % I/O  |
|-----|------------------|---------------------|--------------------|------------------------|---------|--------|
| 2   | $60,51 \pm 5,20$ | $30,85 \pm 13,23$   | $177,51 \pm 29,73$ | $8\,971,23 \pm 323,48$ | 1,00×   | 0,34%  |
| 4   | $51,28 \pm 2,82$ | $78,23 \pm 27,79$   | $212,76 \pm 19,33$ | $4\,823,20 \pm 40,39$  | 1,86×   | 1,62%  |
| 8   | $18,94 \pm 3,76$ | $201,67 \pm 53,32$  | $174,04 \pm 12,90$ | $2\,639,18 \pm 32,59$  | 3,40×   | 7,64%  |
| 16  | $4,52 \pm 0,33$  | $527,94 \pm 53,07$  | $167,77 \pm 7,07$  | $1\,873,56 \pm 52,28$  | 4,79×   | 28,18% |
| 32  | $3,37 \pm 0,10$  | $996,57 \pm 166,98$ | $304,96 \pm 3,83$  | $2\,013,06 \pm 25,99$  | 4,46×   | 49,51% |

### 4.2.2 Avaliação da implementação MPI-IO

Esta subseção analisa o desempenho da arquitetura proposta, baseada em escritas independentes com MPI-IO sobre o sistema de arquivos Lustre, no Experimento AWS. Nesta abordagem cada processo MPI escreve seus próprios dados diretamente no FSx for Lustre, eliminando o ponto de contenção associado a um único *rank* escritor e explorando o paralelismo dos múltiplos OSTs.

A Tabela 4.4 consolida os resultados obtidos com a implementação MPI-IO no Experimento AWS.

Tabela 4.4: Desempenho do Experimento AWS com a implementação MPI-IO, incluindo tempos de I/O, comunicação, tempo total, *speedup* e percentual de I/O.

| Nós | Banda<br>(Gb/s)    | I/O<br>(s)        | Comunicação<br>(s) | Total<br>(s)           | Speedup | % I/O |
|-----|--------------------|-------------------|--------------------|------------------------|---------|-------|
| 2   | $33,80 \pm 3,21$   | $71,89 \pm 68,92$ | $170,21 \pm 17,52$ | $8\,847,89 \pm 127,16$ | 1,00×   | 0,81% |
| 4   | $52,22 \pm 9,25$   | $31,30 \pm 6,94$  | $188,12 \pm 14,94$ | $4\,795,86 \pm 18,74$  | 1,84×   | 0,65% |
| 8   | $83,49 \pm 12,71$  | $26,33 \pm 5,28$  | $206,21 \pm 18,68$ | $2\,574,70 \pm 31,83$  | 3,44×   | 1,02% |
| 16  | $96,17 \pm 8,31$   | $20,37 \pm 3,98$  | $215,97 \pm 33,65$ | $1\,613,30 \pm 45,80$  | 5,48×   | 1,26% |
| 32  | $142,98 \pm 21,95$ | $18,18 \pm 3,54$  | $269,76 \pm 18,83$ | $1\,374,10 \pm 61,95$  | 6,44×   | 1,32% |

### 4.2.3 Análise comparativa

Os resultados das duas subseções anteriores são consolidados visualmente nas figuras a seguir, que apresentam, lado a lado, os perfis de banda agregada, tempo de execução e tempo de bloqueio de envio em função do número de nós para as duas implementações.

A Figura 4.5 apresenta a evolução da largura de banda agregada média no Experimento AWS. As duas curvas exibem comportamentos qualitativamente opostos: a implementação atual parte de aproximadamente 45,82 Gb/s em 2 nós e degrada-se de forma acentuada com o aumento do paralelismo, atingindo apenas 16,62 Gb/s em 8 nós, 2,49 Gb/s em 16 nós e 1,83 Gb/s em 32 nós — uma redução de mais de uma ordem de grandeza. Em contraste, a implementação MPI-IO parte de 26,99 Gb/s em 2 nós e cresce monotonicamente, alcançando 44,41 Gb/s em 4 nós, 74,60 Gb/s em 8 nós, 133,91 Gb/s em 16 nós e 215,50 Gb/s em 32 nós.

É interessante notar que, em 2 nós, a banda agregada da implementação atual é cerca de 70% maior do que a da implementação MPI-IO. Esse comportamento é coerente com o fato de que, em pequena escala, o gargalo do processo escritor único ainda não se manifesta, e o caminho de escrita atual — que utiliza o adaptador de rede do nó mestre de forma exclusiva — aproveita melhor a infraestrutura disponível do que operações independentes coordenadas pela camada MPI-IO, que ainda têm

custo fixo elevado quando há poucos processos. A partir de 4 nós, no entanto, a curva da implementação MPI-IO já alcança a atual, e em 8 nós passa a ser aproximadamente  $4,5\times$  superior. Em 32 nós, a implementação MPI-IO apresenta banda agregada cerca de  $118\times$  maior que a atual, indicando que a arquitetura proposta é capaz de explorar a infraestrutura de armazenamento em uma escala completamente diferente da arquitetura atual.

Essa diferença de comportamento decorre diretamente da eliminação do processo escritor único. Na arquitetura atual, todas as escritas convergem para o adaptador de rede de um único nó, que rapidamente se torna ponto de contenção e limita o *throughput* global — explicando a queda contínua da banda observada. Na abordagem MPI-IO, as operações de escrita são distribuídas entre os processos MPI, permitindo que múltiplos fluxos de dados sejam enviados simultaneamente ao sistema de arquivos Lustre, explorando o paralelismo proporcionado pelos múltiplos OSTs. Como consequência, a banda agregada escala aproximadamente de forma proporcional ao número de nós, demonstrando que o sistema de armazenamento ainda não atingiu seu limite nominal mesmo na maior configuração avaliada.

Adicionalmente, observa-se que os desvios padrão da implementação MPI-IO permanecem relativamente reduzidos em todas as escalas, indicando alta previsibilidade do sistema. Já a implementação atual apresenta desvios padrão da mesma ordem de grandeza da própria banda média a partir de 16 nós, evidenciando comportamento errático típico de regime saturado. Dessa forma, a figura evidencia que a adoção de MPI-IO não apenas elimina o gargalo da arquitetura atual, mas também permite um uso significativamente mais eficiente e estável da largura de banda disponível, tornando a aplicação adequada para execuções em larga escala.

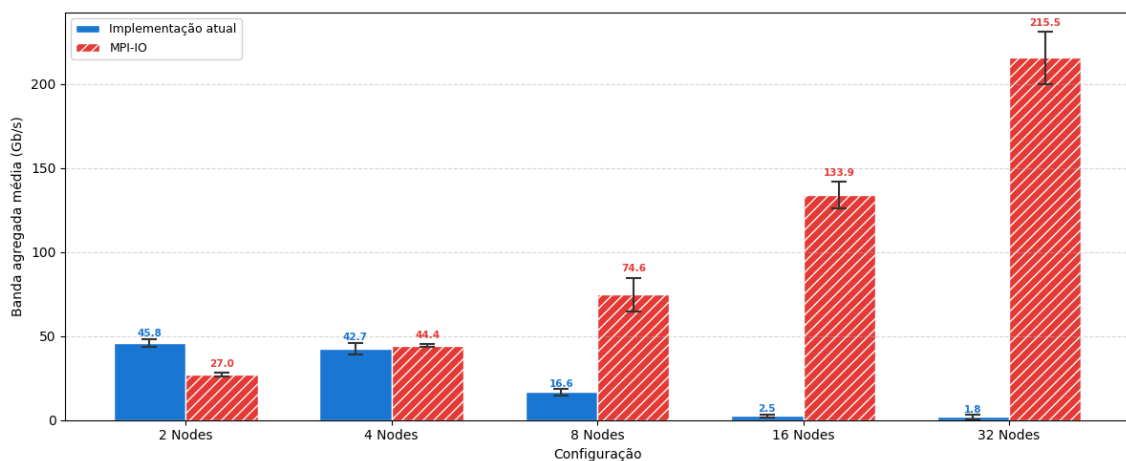


Figura 4.5: Banda agregada média no Experimento AWS.

A Figura 4.6 apresenta a evolução do tempo total de execução no Experimento AWS, contrastando a implementação atual e a implementação MPI-IO sobre o Lustre. Nas configurações de menor escala, em que a E/S não é o componente domi-

nante, as duas curvas evoluem de forma muito próxima: em 2 nós,  $9,55 \times 10^3$  s no modelo atual contra  $9,23 \times 10^3$  s no modelo MPI-IO (ganho de aproximadamente 3%); em 4 nós,  $4,87 \times 10^3$  s contra  $4,82 \times 10^3$  s (ganho de cerca de 1%). Esse comportamento é coerente, uma vez que, em pequena escala, o tempo de execução é dominado pela computação e a contenção sobre o processo escritor ainda não se manifesta.

A divergência entre as duas curvas torna-se evidente a partir de 8 nós e se acentua de forma marcante nas configurações de maior porte. Enquanto a implementação atual apresenta um regime de saturação — com o tempo de execução estabilizando em torno de  $2,6 \times 10^3$  s em 16 nós e voltando a crescer para aproximadamente  $2,97 \times 10^3$  s em 32 nós —, a implementação MPI-IO mantém uma redução consistente do tempo total, atingindo  $1,71 \times 10^3$  s em 16 nós e  $1,39 \times 10^3$  s em 32 nós. Em termos relativos, isso corresponde a uma redução do tempo total de cerca de 35% em 16 nós e de 53% em 32 nós em relação ao modelo atual, evidenciando que a distribuição das operações de E/S elimina o gargalo associado ao processo escritor único e permite que a aplicação continue se beneficiando do aumento do paralelismo.

A figura também evidencia que, na implementação atual, o crescimento do número de nós deixa de produzir ganho de desempenho a partir de 16 nós e passa a ser, na prática, contraproducente em 32 nós — comportamento característico de regime degradado sob *strong scaling*, no qual a contenção do processo escritor mais do que compensa os ganhos de paralelismo computacional. Em contraste, a curva MPI-IO permanece monotonicamente decrescente em todo o intervalo avaliado, indicando que a infraestrutura de armazenamento ainda não atingiu sua capacidade limite mesmo na maior configuração testada.

Cabe observar que, na implementação MPI-IO, parte do tempo total passa a ser ocupada pelas operações coletivas de abertura e fechamento de arquivos. Embora essas operações ocorram apenas no início e no término da execução, elas impõem sincronização global entre todos os processos participantes, introduzindo um custo fixo decorrente da coordenação coletiva. Como foram simulados apenas três estágios por cenário, esse *overhead* não é suficientemente amortizado ao longo da execução, tornando sua contribuição relativa mais evidente no tempo total observado. Em cenários com maior número de estágios, esse mesmo custo tende a ser diluído, passando a representar uma fração menor do tempo total de execução. Ainda assim, mesmo com esse *overhead* presente, a implementação MPI-IO apresenta tempo total inferior ao da implementação atual em todas as escalas avaliadas, e a vantagem cresce com o número de nós.

A Figura 4.7 apresenta o tempo médio de bloqueio das operações de E/S, em segundos, em função do tamanho das mensagens no Experimento AWS, utilizando MPI-IO sobre o sistema de arquivos Lustre, em escala logarítmica. Observa-se que o

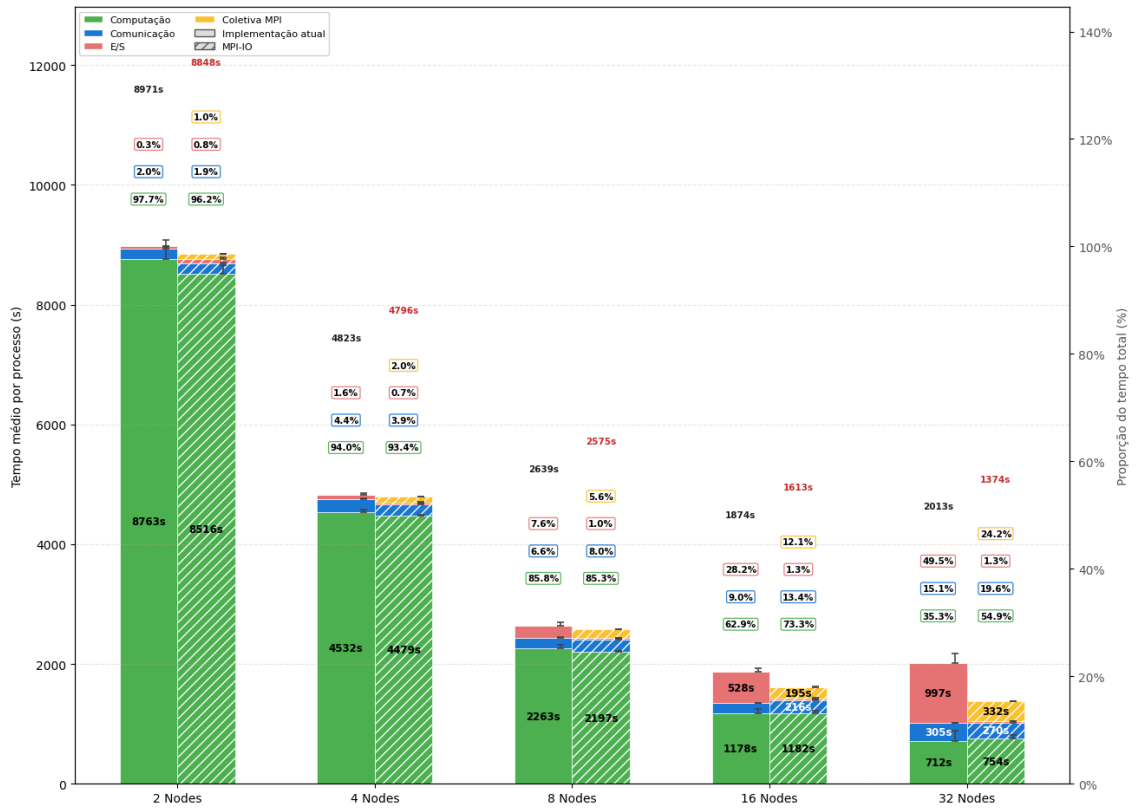


Figura 4.6: Tempo de execução no Experimento AWS.

tempo de bloqueio apresenta crescimento com o aumento do volume de dados, refletindo o custo associado à interação com o sistema de arquivos paralelo. No entanto, esse crescimento ocorre de forma mais controlada quando comparado à abordagem baseada em comunicação ponto a ponto, indicando uma redução significativa dos efeitos de contenção. A utilização da escala logarítmica permite evidenciar a variação em diferentes ordens de magnitude, destacando que, mesmo para mensagens maiores, o tempo de bloqueio permanece relativamente limitado. Esses resultados indicam que a estratégia baseada em escritas independentes com MPI-IO permite melhor amortização dos custos de E/S, contribuindo para maior eficiência e escalabilidade da aplicação.

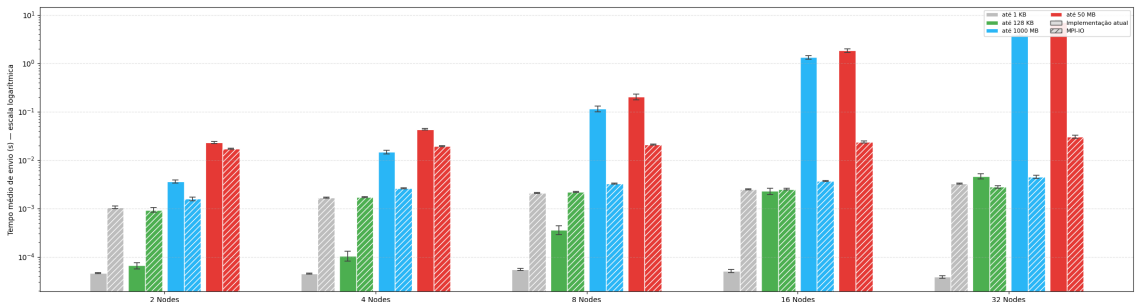


Figura 4.7: Tempo médio de bloqueio das operações de envio em função do tamanho das mensagens no Experimento AWS (escala logarítmica).

#### 4.2.4 Avaliação do impacto do número de OSTs

Esta subseção tem como objetivo avaliar, no Experimento AWS, o impacto do número de *Object Storage Targets* (OSTs) no desempenho das operações de E/S no sistema de arquivos Lustre. Em ambientes HPC, a quantidade de OSTs está diretamente relacionada ao grau de paralelismo de E/S disponível, uma vez que os dados podem ser distribuídos entre múltiplos dispositivos de armazenamento, permitindo acessos concorrentes por diferentes processos.

Para esta avaliação, foram realizados experimentos variando-se a quantidade de OSTs disponíveis, mantendo-se constantes as demais configurações do ambiente, como número de nós de cálculo, volume de dados e padrão de acesso. O objetivo foi isolar o efeito da infraestrutura de armazenamento sobre o desempenho global da aplicação.

Os resultados obtidos indicam que a redução no número de OSTs impacta negativamente o desempenho das operações de E/S, uma vez que diminui o paralelismo efetivo no acesso aos dados. Com menos OSTs disponíveis, ocorre maior contenção de acesso, levando a um aumento no tempo de execução das operações de escrita e leitura. Por outro lado, um maior número de OSTs possibilita melhor distribuição dos dados e maior aproveitamento da largura de banda agregada do sistema de arquivos.

Esses resultados reforçam a importância do correto dimensionamento da infraestrutura de armazenamento em ambientes baseados em Lustre, especialmente para aplicações intensivas em E/S. A escolha inadequada do número de OSTs pode introduzir gargalos significativos, limitando a escalabilidade da aplicação mesmo na presença de recursos computacionais suficientes.

A Figura 4.8 apresenta os resultados do experimento conduzido com diferentes quantidades de nós OST, com o objetivo de avaliar o impacto direto da infraestrutura de armazenamento no desempenho da aplicação. Diferentemente da avaliação apresentada nas seções anteriores, nesta etapa foi utilizada a abordagem de E/S baseada em escritas independentes com MPI-IO, permitindo isolar de forma mais clara o efeito do paralelismo do sistema de arquivos Lustre sobre o tempo de execução.

Os resultados evidenciam que a variação no número de OSTs impacta diretamente o desempenho observado, uma vez que altera o grau de paralelismo disponível para as operações de E/S. Com um maior número de OSTs, observa-se melhor distribuição dos dados e redução da contenção de acesso, enquanto configurações com menor quantidade de OSTs tendem a concentrar as operações, resultando em aumento do tempo total de execução. Dessa forma, a figura reforça a relevância do dimensionamento adequado dos recursos de armazenamento para garantir a escalabilidade da solução proposta.

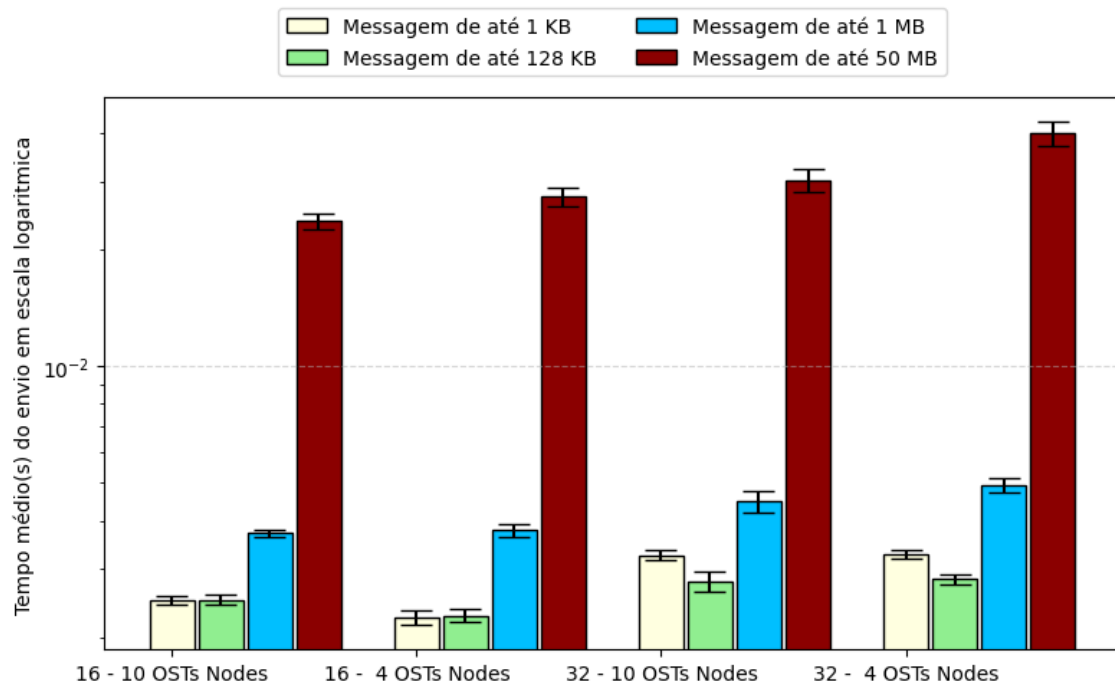


Figura 4.8: Experimento com diferentes nós OSTs.

A Figura 4.9 apresenta a largura de banda agregada média para diferentes configurações de número de nós e quantidade de OSTs no sistema de arquivos Lustre. Observa-se que, para ambas as configurações de 16 e 32 nós, a variação no número de OSTs (10 e 4) não provoca uma degradação significativa na banda agregada média.

Para 16 nós, os valores de banda permanecem praticamente constantes entre as configurações com 10 e 4 OSTs, indicando que, nesse nível de paralelismo, o sistema de armazenamento ainda não se encontra saturado. Já para 32 nós, embora a configuração com 10 OSTs apresente banda ligeiramente superior, a diferença observada é moderada, sugerindo que o sistema continua operando de forma eficiente mesmo com menor número de dispositivos de armazenamento.

Esses resultados indicam que, do ponto de vista da banda agregada, a aplicação consegue explorar adequadamente o paralelismo disponível no sistema de arquivos, mesmo com uma redução no número de OSTs. No entanto, essa análise deve ser complementada com métricas de tempo de execução, uma vez que a banda média isoladamente pode não capturar possíveis impactos relacionados à contenção, latência ou sobrecarga de metadados.

Dessa forma, embora a redução do número de OSTs não comprometa significativamente a banda agregada neste experimento, sua influência sobre o tempo total de execução e a eficiência global do sistema deve ser analisada de forma conjunta.

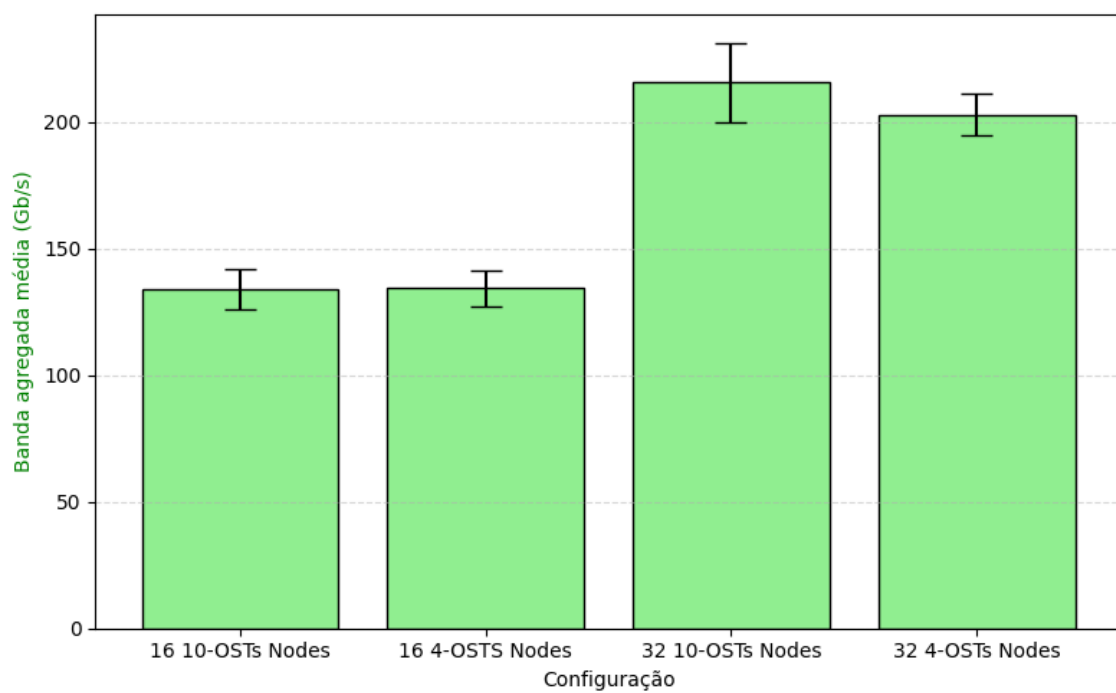


Figura 4.9: Banda agregada média no experimento de OSTs.

# Capítulo 5

## Conclusões

Os resultados apresentados nesta dissertação evidenciam que o desempenho da aplicação SDDP é fortemente influenciado pela estratégia adotada para as operações de entrada e saída (E/S). A análise detalhada do comportamento do `MPI_Send` revelou que o tempo de bloqueio associado ao envio de mensagens representa um dos principais fatores limitantes da escalabilidade do modelo, sobretudo quando há um único processo responsável pela escrita em disco. Esse gargalo torna-se mais pronunciado à medida que cresce o número de nós computacionais, restringindo a sobreposição entre comunicação e computação e, conseqüentemente, a eficiência paralela da aplicação.

A adoção de uma arquitetura de E/S MPI-IO, baseada em MPI-IO e no sistema de arquivos Lustre, mostrou-se uma alternativa promissora para mitigar esses efeitos. Essa abordagem permite que múltiplos processos MPI realizem operações de escrita de forma paralela e coordenada, eliminando a dependência de um processo escritor único e aproveitando o paralelismo nativo do sistema de arquivos. Além de reduzir o tempo de bloqueio dos processos, essa solução potencializa o uso da largura de banda agregada e melhora a escalabilidade em aplicações com alto volume de dados.

Como trabalhos futuros, propõe-se investigar o impacto de diferentes estratégias de agregação de escrita coletiva (*two-phase I/O*) e o uso de *hints* do ROMIO para otimizar o alinhamento das operações de escrita com a configuração de *striping* do Lustre. Tais aprimoramentos poderão contribuir para a consolidação de um modelo de E/S escalável e portátil, aplicável a uma ampla gama de aplicações científicas paralelas em ambientes de HPC e nuvem.

# Referências Bibliográficas

- TODO: AUTOR. “TODO: título da referência sobre paralelismo em ciência e engenharia”. 2024. Placeholder — substituir pela referência correta.
- THAKUR, R., GROPP, W., LUSK, E. “On Implementing MPI-IO Portably and with High Performance”, *Proc. of the 6th Workshop on I/O in Parallel and Distributed Systems*, pp. 23–32, 1999.
- TODO. “TODO: trabalho sobre MPI-IO / Lustre / escalabilidade”, 2010a.
- BRAAM, P. J. “The Lustre Storage Architecture”, *Cluster File Systems Inc.*, 2002.
- TODO. “TODO: integração de fontes renováveis em sistemas elétricos”, 2010b.
- TODO. “TODO: variabilidade/incerteza de fontes renováveis”, 2019.
- PEREIRA, M. V. F., PINTO, L. M. V. G. “Multi-stage Stochastic Optimization Applied to Energy Planning”, *Mathematical Programming*, v. 52, n. 1–3, pp. 359–375, 1991.
- MESSAGE PASSING INTERFACE FORUM. “MPI: A Message-Passing Interface Standard, Version 4.1”. 2023. <https://www.mpi-forum.org/>.
- TODO. “TODO: protocolos eager/rendezvous em MPI”. In: *TODO*, 1999.
- TODO. “TODO: análise de custo de comunicação MPI”. In: *TODO*, 2005a.
- MESSAGE PASSING INTERFACE FORUM. “MPI-2: Extensions to the Message-Passing Interface”. 1997. Specification.
- TODO. “Birds-of-a-Feather: Lustre in the Top500”. SC’25, 2025. [https://sc25.supercomputing.org/proceedings/bof/bof\\_pages/bof197.html](https://sc25.supercomputing.org/proceedings/bof/bof_pages/bof197.html).
- TODO. “TODO: comparação Ethernet x InfiniBand”, 2003.
- TODO. “TODO: RDMA / InfiniBand em HPC”. In: *TODO*, 2005b.
- TODO. “TODO: HPC em nuvem”. In: *TODO*, 2010c.

- TODO. “TODO: AWS / FSx for Lustre / EFA para HPC”, 2020.
- OPERADOR NACIONAL DO SISTEMA ELÉTRICO. “Programa Mensal da Operação — PMO”. <https://www.ons.org.br/>, 2023.
- TODO. “TODO: granularidade de operações de E/S e desempenho”. In: *TODO*, 2009.
- NIKOLOPOULOS, D. S. “Dynamic Tiling for Effective Use of Shared Caches on Multithreaded Processors”, *International Journal of High Performance Computing and Networking*, v. 2, n. 1, pp. 22–35, 2004. doi: 10.1504/IJHPCN.2004.009265.
- AMDAHL, G. M. “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities”. In: *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, AFIPS ’67 (Spring), pp. 483–485, New York, NY, USA, 1967. Association for Computing Machinery. doi: 10.1145/1465482.1465560.
- INTEL CORPORATION. “Intel Xeon Scalable Processors — 4th Generation (Sapphire Rapids)”. <https://www.intel.com/content/www/us/en/products/details/processors/xeon/scalable.html>, 2023.
- AMAZON WEB SERVICES. “Amazon EC2 C7i Instances — Compute Optimized”. <https://aws.amazon.com/ec2/instance-types/c7i/>, 2024.
- OPENSFS AND EOFS. *Lustre Software Release 2.x: Operations Manual*, 2025. [https://doc.lustre.org/lustre\\_manual.pdf](https://doc.lustre.org/lustre_manual.pdf).